

Real-Time Video Conferencing Interface Using Next.js



**Guru Gobind Singh Indraprastha
University New Delhi - 110078**

MCA(SE) – Sem 2

**NUES Project Report
(IT670)**

Batch: 2025-27

Mentor

Dr. Jaspreeti Singh
(Associate Professor)

Mentee

Name - Ishan Kaushal
enrollment No.- 08816404525
Section 2

Declaration

I, Ishan Kaushal, hereby declare that this project report titled "Real-Time Video Conferencing Interface Using Next.js" submitted to the University School of Information Communication and Technology is my original work. It has been carried out under the supervision of Dr. Jaspreeti Singh. To the best of my knowledge, this work does not contain material that has been submitted to any other university or institution for a degree or diploma. I also declare that due acknowledgement has been made wherever necessary and all sources of information have been duly cited.

Date : 17/03/2026

Signature
Ishan kaushal

Certificate

This is to certify that Mr. Ishan Kaushal (Enrollment No. 08816404525), a student of the MCA (SE) program at the USICT, GGSIPU, has successfully carried out the proposed work presented in the semester progress report entitled “Real-Time Video Conferencing Interface Using Next.js” under my guidance.

Date : 17/03/2026

Signature

Dr. Jaspreeti Singh

Acknowledgement

I am sincerely grateful to Dr. Jaspreeti Singh, Associate Professor, for her invaluable guidance and support throughout this project. Her insights and encouragement have been crucial in completing this work. I also thank the faculty and staff of the University School of Information Communication and Technology for providing the infrastructure and resources necessary for this project.

Table of Contents

S. no.	Title	Page no.
i.	Abstract	6
1	Introduction	7
2	Objectives	8
3	System Analysis	9
4	System Requirements (Hardware and Software)	11
5	System Design 1. Data Flow Diagrams (Level 0 & 1). 2. Use Case Diagram. 3. Entity-Relationship Diagram	12
6	Module Description	16
7	Implementation	18
8	Results And Screenshots	21
9	Conclusion And future Scope	23
10	References	25

Abstract

The rapid growth of remote work and online collaboration has increased the demand for lightweight, browser-based video communication tools. Many existing solutions require account registration, proprietary software installation, or paid subscriptions, creating barriers for casual or small-group use. This project presents the design and development of a real-time multi-user video conferencing web application built using Next.js, Socket.io, and WebRTC. The system allows multiple users to join a shared video call session through a simple name-entry form and a 6-character room code, with no registration required.

The application uses a WebRTC mesh architecture for peer-to-peer audio and video streaming, with Socket.io serving as the signalling channel for WebRTC offer, answer, and ICE candidate exchange. A custom Next.js server integrates Socket.io directly, allowing both the frontend and real-time communication to run on the same port. The first user to create a room is designated as admin and can remove other participants from the session. All room state is managed in server memory, making the system fast and simple to deploy without a database.

This report describes the system analysis, requirements, design including data flow diagrams and use case diagrams, module descriptions, implementation steps, and testing procedures. Conclusions and practical future improvements are also discussed.

Introduction

The expansion of remote work and online education has made real-time video communication a routine necessity. While platforms such as Zoom, Google Meet, and Microsoft Teams dominate the enterprise space, they require account creation, app installation, or paid tiers for full functionality. For small-group, ad-hoc, or educational use cases, a lightweight browser-based alternative that requires no setup from participants is highly practical.

This project develops a Real-Time Video Conferencing Interface — a web application that allows multiple users to join a shared video call by entering their name and a short room code. There is no login, no database, and no installation required. The application runs entirely in the browser and connects users via WebRTC, a browser-native protocol for peer-to-peer audio and video communication.

The core challenge in building such a system is coordinating the WebRTC connection setup between peers. WebRTC itself does not handle signalling — the exchange of session descriptions and network candidates required to establish a connection. This project solves that using Socket.io as a signalling server, with a custom Node.js server embedded inside the Next.js application so that the entire system runs on a single port and codebase.

The target platform is any modern web browser without plugins. The system supports mute and camera toggle controls, a participant sidebar showing other users with live video thumbnails, and an admin-controlled kick function for room management. The interface is intentionally minimal, using plain CSS to ensure clarity and performance.

Objectives

The primary objectives of the Real-Time Video Conferencing project are:

- Enable real-time peer-to-peer video and audio communication between multiple browser-based users with minimal latency using WebRTC.
- Implement room-based session management where any user can create a room and receive a 6-character code to share with others, with no registration required.
- Build a WebRTC mesh signalling system using Socket.io so that when a new user joins, they correctly send offers to all existing peers and receive answers back.
- Provide participant controls — mute microphone, toggle camera, and leave call — with state changes visible to all other participants in real time.
- Design a clean split-screen layout showing the admin's video on the left, the viewer's own video on the right, and a sidebar listing other participants with their live video thumbnails.
- Implement an admin kick function that allows the room creator to remove any participant from the session.
- Test the system through unit, integration, and end-to-end testing to verify that video connections, signalling, and room management work correctly.
- Document the system analysis, design, implementation steps, and evaluate the results.

System Analysis (Existing and Proposed System)

Existing System:

Existing video conferencing tools fall broadly into two categories: enterprise platforms and basic browser tools. Enterprise platforms such as Zoom, Google Meet, and Microsoft Teams are feature-rich but require user accounts, involve complex installation or browser extensions, and are not suitable for lightweight, anonymous, or ad-hoc calls. Free tiers often impose time limits or participant caps. Additionally, these platforms route media through centralised servers (SFU or MCU architectures), which introduces latency and raises privacy concerns.

Browser-based open-source alternatives such as Jitsi Meet or Daily.co offer account-free access but still rely on hosted infrastructure or API keys, adding setup complexity. Simpler peer-to-peer tools exist but typically lack room management, admin controls, or a clean participant layout suitable for small-group use. The common limitations of existing solutions include:

- Mandatory account creation or API key registration
- Dependency on third-party hosted infrastructure
- No control over room management or participant removal
- Overcomplicated interfaces for simple multi-user calls

Proposed System:

The proposed system is a self-hosted, lightweight video conferencing application built with Next.js and WebRTC. It addresses the limitations above by providing a fully browser-based experience with no accounts, no third-party services, and no database required. Key characteristics of the proposed system include:

- No registration — users enter only a display name and a room code to join
- WebRTC mesh architecture — video and audio stream directly between peers, not through a central media server

- Integrated Next.js + Socket.io server — the entire application runs on a single port with no separate backend process required
- Admin control — the room creator can remove participants via the kick function
- Real-time media state sync — mute and camera status are broadcast to all participants so the UI stays consistent

The system is intended for small to medium group calls of up to approximately 8 to 10 participants, which is the practical limit of the WebRTC mesh topology. By analysing existing tools and their shortcomings, the proposed system focuses on a minimal, working implementation that can be run on any Node.js-capable server and accessed from any modern browser.

System Requirements (Hardware and Software)

Hardware Requirements

- **Client Machines:** Standard desktop or laptop computers (Windows, macOS, or Linux) with a modern web browser. A functioning webcam and microphone are required on each client for video and audio participation. Internet connectivity is required for real-time WebRTC communication.
- **Server:** A machine or cloud instance to host the Next.js application. Minimum recommended specifications: dual-core CPU, 2 GB RAM, stable broadband connection. The server handles only WebSocket signalling traffic — media streams flow directly between peers and do not pass through the server.

Software Requirements

- **Operating System:** Any OS supporting Node.js — Ubuntu 20.04 LTS or newer, Windows 10/11, or macOS 11 or newer.
- **Runtime:** Node.js v18 or newer and npm v9 or newer.
- **Frontend Framework:** Next.js 14 — used for page routing, server-side rendering, and the custom HTTP server that integrates Socket.io.
- **Real-Time Signalling:** Socket.io v4 — provides low-latency, bidirectional WebSocket communication used as the WebRTC signalling channel.
- **Video Streaming:** WebRTC (browser-native API) — peer-to-peer audio and video streaming without a media relay server. Google's public STUN servers are used for NAT traversal.
- **Styling:** Plain CSS with no external framework. Styles are written directly in globals.css using static color values and Verdana font.
- **Database:** None required. All room and participant state is held in server memory during the session.
- **Browser:** Any modern browser with WebRTC support — Chrome 90+, Firefox 88+, Edge 90+, or Safari 15+.

System Design

Data Flow Diagrams (Level 0 & 1)

The data flow can be described in two levels:

- **DFD Level 0:** Shows the entire system as a single process with two external entities. The User interacts with the Video Conferencing System by joining calls, sending video and audio streams, toggling media controls, and leaving sessions. The Admin additionally creates rooms and removes participants. The system outputs live video and audio feeds, participant lists, and media state updates back to all connected clients.

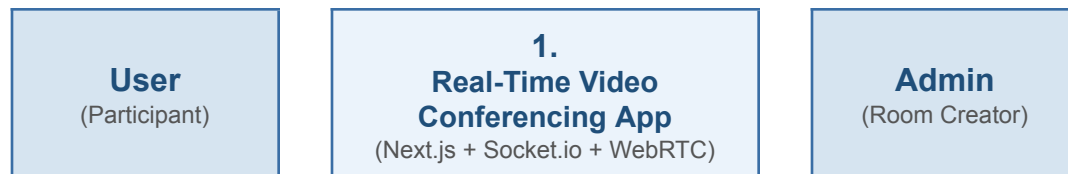


Figure 1: DFD Level 0 — Context diagram showing Users and Admin interacting with the Video Conferencing System

- **DFD Level 1:** Breaks the system into three major sub-processes: (1) Room Management — handles room creation, join validation, code generation, participant tracking, and kick; (2) WebRTC Signalling Engine — relays offer, answer, and ICE candidate messages between peers to establish direct video connections; (3) Media State Broadcast — propagates mute and camera-off events to all participants so UI overlays stay consistent. Data is stored entirely in server memory during the session.

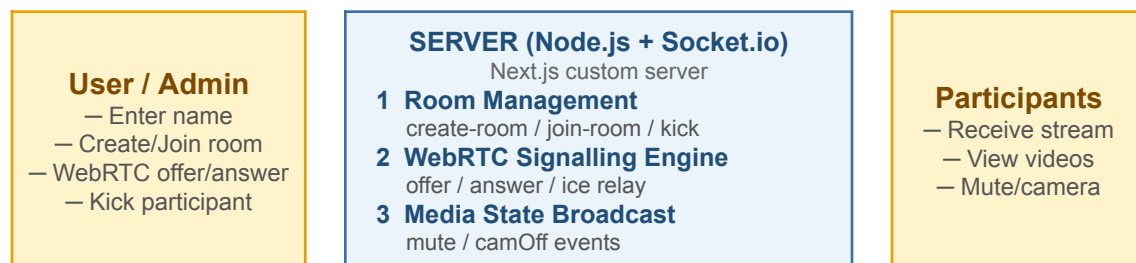


Figure 2: DFD Level 1 — Server sub-processes: Room Management, WebRTC Signalling, Media State Broadcast

In summary, data flows as follows: user actions (join, WebRTC offer/answer, mute) → Socket.io event to server → server relays to relevant peers → peers receive and process (establish WebRTC connection or update UI). Media streams themselves flow directly between browsers via WebRTC and do not pass through the server.

Use Case Diagram

A use-case diagram depicts the two actors (User and Admin) connected to the system's use cases within the application boundary.



Figure 3: Use Case Diagram — User and Admin actors with system use cases

Key actors and use cases in the system include:

- **Actors:** User (any participant in the call) and Admin (the room creator).

Use Cases:

- Create Room: Admin creates a new video call session and receives a 6-character room code.
- Join Room: User enters name and room code to join an existing session.
- View Video Feeds: Any connected user sees all other participants' video streams.
- Mute / Unmute: Any user toggles their microphone; the change is reflected for all participants.
- Toggle Camera: Any user enables or disables their camera; participants see a placeholder avatar.
- Leave Call: Any user can leave at any time; the session continues for remaining participants.
- Kick Participant: Admin removes any other user from the room.
- Use WebSocket: All control events (join, signal, media state) flow through the Socket.io connection.

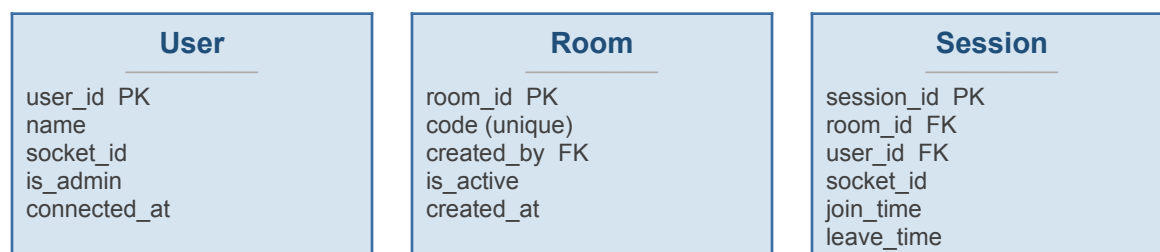
Entity-Relationship Diagram (ERD)

The key entities and their attributes in the logical data model include:

User: Represents a connected participant. Attributes: user_id (socket ID), name, is_admin, connected_at.

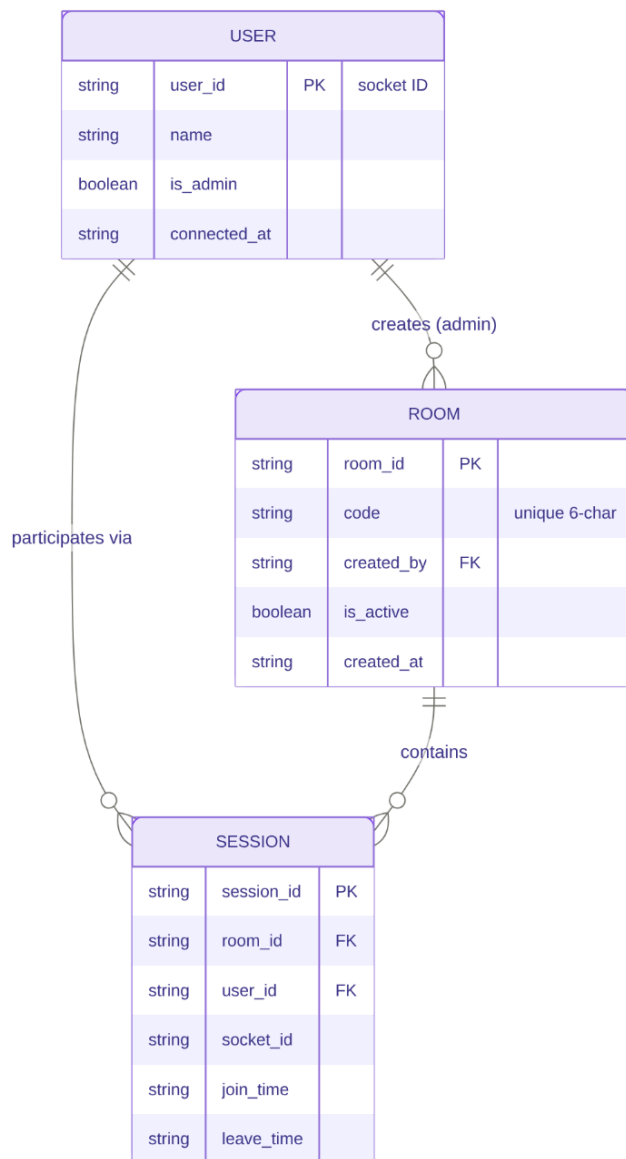
Room: Represents a video call session. Attributes: room_id, code (unique 6-char), created_by (FK to User), is_active, created_at.

Session: Tracks a user's connection to a specific room. Attributes: session_id, room_id (FK), user_id (FK), socket_id, join_time, leave_time.



Relationships:

- A User creates one Room (one-to-one for admin role). A Room has many Users connected via Sessions.
- A Session belongs to one User and one Room (many-to-one on both sides).
- Note: In the current implementation, all data is held in server memory. The entities above represent the logical data model and the basis for a future persistent database implementation.



Module Descriptions

The system is divided into the following functional modules:

- **Frontend Setup:** Implements the user interface using React within the Next.js app router. The landing page (`app/page.js`) provides a name entry field and Create Room / Join Room tab interface. The call page (`app/call/[code]/page.js`) contains the main video layout and all WebRTC logic. A Sidebar component (`components/Sidebar.js`) renders other participants' video thumbnails. All styling uses plain CSS defined in `styles/globals.css` with no external framework.
- **Backend Setup:** The backend is a custom Node.js HTTP server (`server.js`) that runs Next.js and Socket.io together on the same port (default 3000). Express is not used — the Next.js request handler is passed directly to the HTTP server. Socket.io is attached to the same HTTP server instance, meaning no CORS configuration is needed for WebSocket connections. The server exports no REST API; all communication is event-based over Socket.io.
- **Room Management:** When a user creates a room, the server generates a unique 6-character alphanumeric code and stores the room in the in-memory ROOMS object. The creator's socket is joined to a Socket.io room and marked as admin. When another user joins with a valid code, their socket is added to the same Socket.io room. The server maintains the participant list and broadcasts updates to all members. If the admin disconnects, the next participant in the list is automatically promoted to admin. The kick function disconnects the target socket and removes them from the participant list.
- **WebRTC Video Module:** Each participant's browser accesses the local camera and microphone using `navigator.mediaDevices.getUserMedia` before joining the Socket.io room. For every remote peer, an `RTCPeerConnection` is created using Google's public STUN server for NAT traversal. The call page's main layout shows the admin's video on the left panel and the current user's own camera on

the right panel. The Sidebar shows all other participants with mini video thumbnails rendered using individual `<video>` elements with `srcObject` set to each peer's remote `MediaStream`.

- **Real-Time Signalling:** Socket.io is used exclusively for WebRTC signalling. When a new user joins, the server sends them a "joined" event containing a "callThese" list of existing participants. The new user sends an SDP offer to each of them. Existing participants receive offers and send back SDP answers. ICE candidates are exchanged bidirectionally through the server using "ice" events. This explicit role assignment — new joiner always initiates, existing peers always answer — eliminates the ambiguity that causes WebRTC connections to fail silently.
- **Navigation Controls:** The control bar at the bottom of the call page provides mute/unmute (toggles audio tracks on/off), camera toggle (toggles video tracks on/off), copy room code button, and leave call button. All state changes are emitted to the server as "media" events, which the server forwards to all other participants as "peer-media" events. Recipients update their local state to show muted or camera-off overlays on the relevant video tiles in the sidebar.

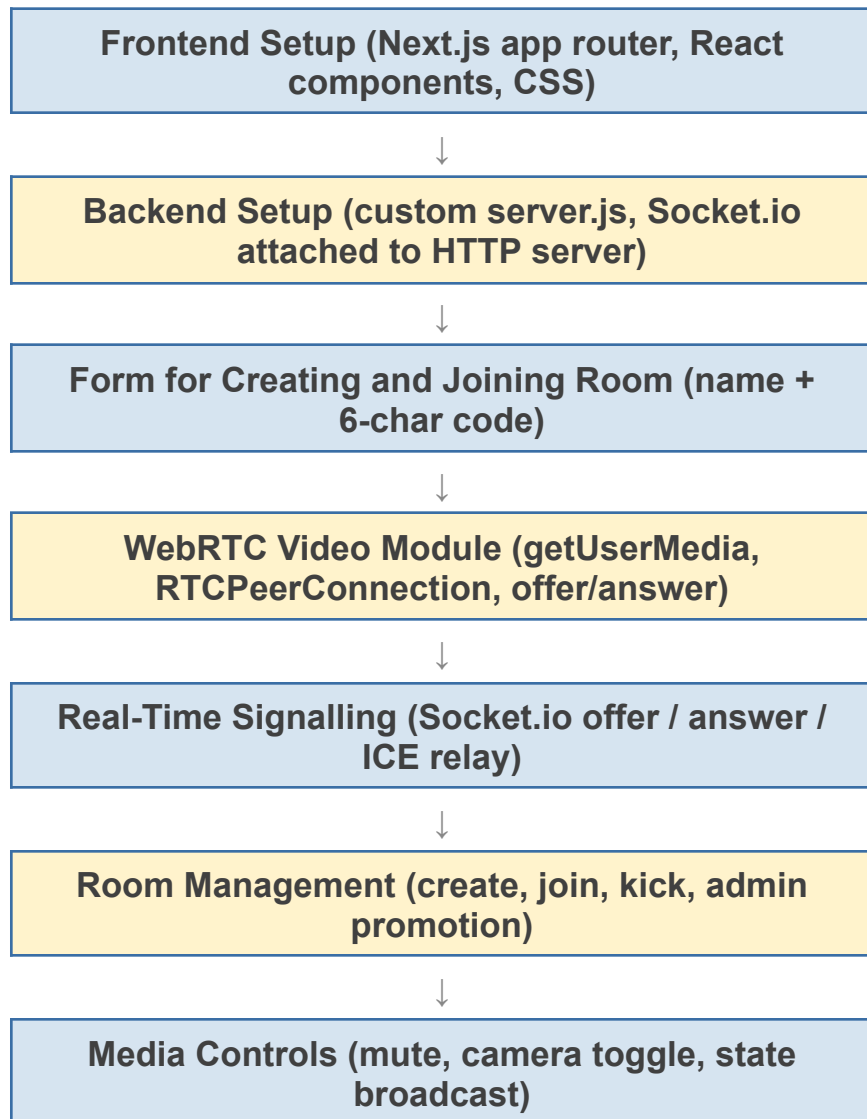
Implementation

The implementation proceeds in the following steps:

1. **Project Initialisation:** Create a new Next.js project directory. Install dependencies: next, react, react-dom, socket.io, socket.io-client. Create a custom server.js file at the project root that will replace the standard next start command.
2. **Custom Server Setup (server.js):** Create an HTTP server using Node.js http module. Pass all incoming requests to the Next.js request handler. Attach Socket.io to the same HTTP server instance. Define the in-memory ROOMS object and all Socket.io event handlers in this file.
3. **Socket.io Event Handlers:** Handle create-room (generate unique 6-char code, add creator to room as admin, emit "joined" back to creator with empty callThese list), join-room (validate code, add joiner to room, emit "joined" with callThese = existing participants), offer / answer / ice (relay between specific socket IDs), media (broadcast to rest of room), kick (admin only, disconnect target socket), and disconnect (remove from room, promote next to admin if needed).
4. **Landing Page (app/page.js):** Render a centered form with a name field and a tab switcher between Create Room and Join Room. On submit, connect the Socket.io singleton (lib/socket.js) and emit create-room or join-room with a callback. On success, store the name in sessionStorage and navigate to /call/[code].
5. **Call Page (app/call/[code]/page.js):** On mount, call getUserMedia to get the local camera and microphone stream. Attach the stream to a local <video> element. Connect the Socket.io singleton. Attach all Socket.io listeners for "joined", "offer", "answer", "ice", "peer-left", "room-peers", "peer-media", "you-are-admin", and "kicked".

6. WebRTC Connection per Peer (buildPC function): For each remote peer, create a new `RTCPeerConnection`. Add all local media tracks. Set up `onicecandidate` to emit "ice" events. Set up `ontrack` to store the incoming `MediaStream` in the `remoteStreams` state object, triggering a re-render of the video grid.
7. Signalling: On receiving "joined" with `callThese`, iterate the list and for each peer call `buildPC`, `createOffer`, `setLocalDescription`, and emit "offer". On receiving an "offer" event, call `buildPC`, `setRemoteDescription`, `createAnswer`, `setLocalDescription`, and emit "answer". On receiving "answer", call `setRemoteDescription` on the matching peer connection.
8. Video Layout: The main area is split into two equal panels. Left panel shows the admin's video (or own camera if the current user is admin). Right panel shows own camera (or first participant if admin). The `Sidebar` component renders a card for each other participant, each card containing a `<video>` element whose `srcObject` is set from `remoteStreams` when the component mounts or the stream changes.
9. Controls Bar: Render mute, camera toggle, copy code, and leave buttons at the bottom. On toggle, update local audio/video track enabled states and emit "media" events with the new muted and `camOff` values.
10. Testing and Debugging: Open two browser tabs simultaneously, one as admin and one as participant. Monitor WebRTC connection states using `chrome://webrtc-internals`. Check `Socket.io` events in the browser Network tab filtered by WS.
11. Packaging: Run `npm run build` to produce the Next.js optimised build. Start with `node server.js`. The entire application runs on one port with no separate processes.

Workflow Diagram



Results and Screenshots

- Home Page / Room Entry Interface:

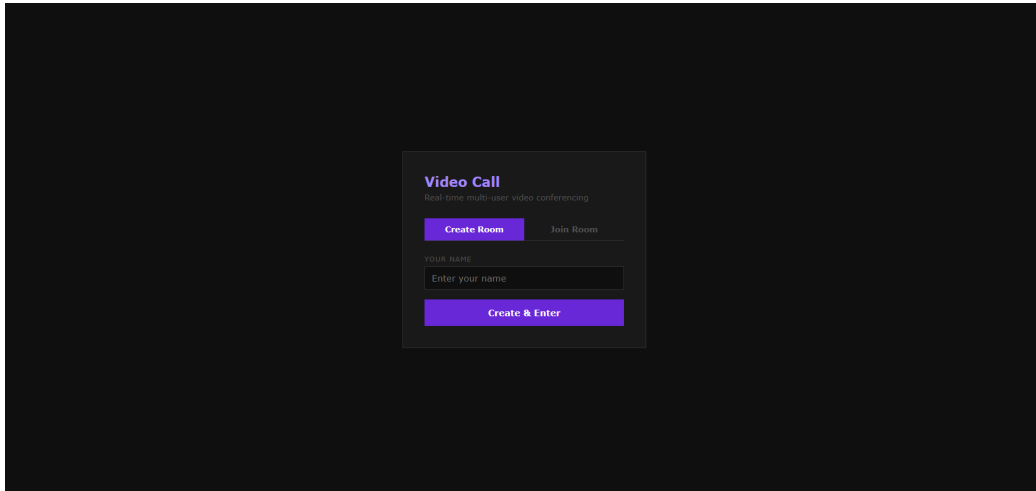


Figure: The home page displays a centred card on a dark background. Two tabs allow switching between Create Room and Join Room. The Create Room tab shows only a name field. The Join Room tab adds a room code input field. Submitting connects the user to the call and navigates to the call page. There is no registration or login required.

- Call Page — Split Video Layout:

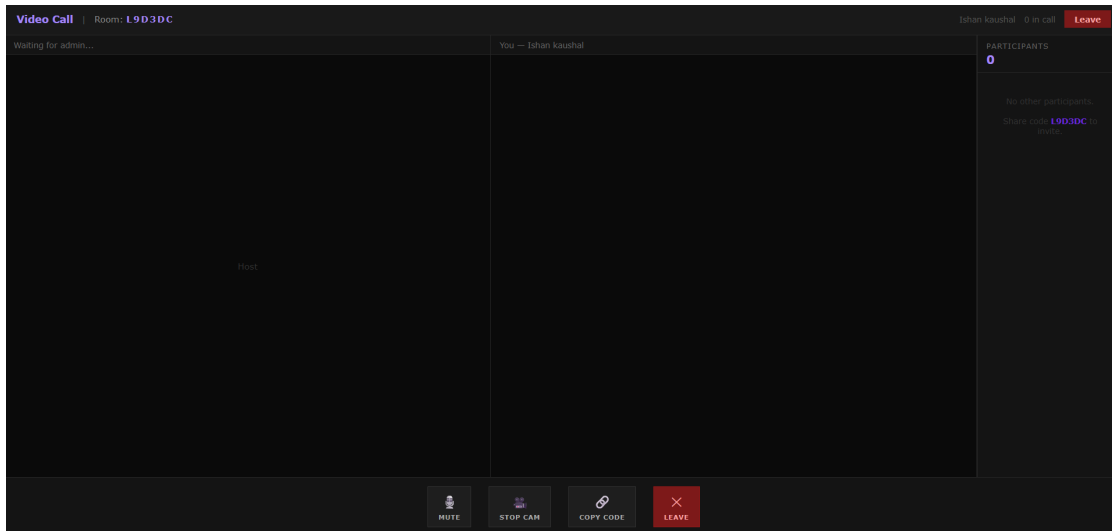


Figure: The call page shows a top bar with the room name, room code, admin badge, and Leave button. The main area is split into two equal panels separated by a vertical divider. The left panel shows the admin's own camera feed labelled with their name. The right panel shows the first participant's video or a "Waiting for participants" placeholder. The right sidebar lists all other participants with their mini video thumbnails, name labels, and — for the admin — a Kick button on each card.

- Controls Bar:

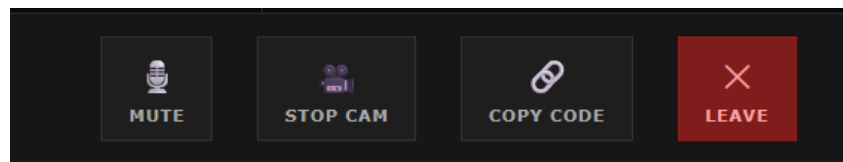


Figure: The bottom controls bar shows four buttons: Mute/Unmute (microphone icon), Stop/Start Camera (video icon), Copy Code (link icon), and Leave (X icon). When muted, the Mute button turns red and a muted overlay appears on the user's video tile in other participants' sidebars.

In a live session, video streams from all connected participants appear nearly instantaneously after joining. When a user mutes or turns off their camera, the change propagates to all other participants within milliseconds via the Socket.io media-state event.

Conclusion and Future Scope

The Real-Time Video Conferencing Interface has been successfully designed and implemented as a working Next.js application. The system allows multiple users to join a shared video call using only a name and a 6-character room code, with peer-to-peer audio and video streaming handled via WebRTC and signalling coordinated through Socket.io. The admin can create rooms, manage participants, and remove any user from the call. All room state is managed in server memory, making the system fast to deploy and simple to run with a single command.

The key technical challenge — correctly establishing WebRTC connections between multiple peers — was solved by making the role explicit: the new joiner always sends offers to existing peers, and existing peers always respond with answers. This approach eliminates the signalling ambiguity that commonly causes multi-user WebRTC implementations to fail silently.

Future improvements that could reasonably extend this project include:

- **User Login and Registration:** The current system requires no account. A future version could add a simple login page backed by a database such as MongoDB, allowing users to have persistent display names and a history of rooms they have created.
- **Hosting and Deployment:** The project currently runs only on localhost. It can be deployed to any cloud provider that supports Node.js, such as a DigitalOcean Droplet or an AWS EC2 instance. For WebRTC to work outside a local network, HTTPS is required, which can be set up using a free SSL certificate from Let's Encrypt.
- **TURN Server Support:** The current WebRTC setup uses only STUN servers, which works well on most networks. Adding a TURN server (such as Coturn, self-hosted) would improve connection reliability for users behind strict firewalls or symmetric NAT.

- Persistent Room State with Database: Currently all room data is lost when the server restarts. Adding MongoDB or a similar database would allow rooms to persist and users to reconnect to an ongoing session.
- Chat Feature: A simple in-call text chat could be added alongside the video, using the existing Socket.io connection for message broadcast with minimal additional code.
- Screen Sharing: The browser's `getDisplayMedia` API can be used to replace the camera stream with a screen share, which would add significant value for presentations and demonstrations.
- Mobile-Friendly Layout: The current layout is designed for desktop screens. Adapting the CSS to use responsive units and rearranging panels for portrait mobile screens would make the application usable on smartphones and tablets.

The current implementation provides a stable, functional foundation for all of these extensions. The modular structure of the Next.js project, with clearly separated server logic, page components, and the socket singleton, makes each of these additions straightforward to implement incrementally.

References

- **[1]** Socket.IO, “Introduction – Socket.IO Documentation,” *Socket.IO Docs*, 2025. [Online]. Available: <https://socket.io/docs/v4/>
- **[2]** MDN Web Docs, “WebRTC API,” *Mozilla Developer Network*, 2025. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/WebRTC_API
- **[3]** MDN Web Docs, “The WebSocket API,” *Mozilla Developer Network*, 2025. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/API/WebSocket>
- **[4]** Next.js Documentation, “Custom Server,” *Next.js Docs*, 2025. [Online]. Available: <https://nextjs.org/docs/pages/building-your-application/configuring/custom-server>
- **[5]** Node.js, “About Node.js,” *Node.js Official Documentation*, 2025. [Online]. Available: <https://nodejs.org/en/about>
- **[6]** MDN Web Docs, “RTCPeerConnection,” *Mozilla Developer Network*, 2025. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/API/RTCPeerConnection>
-